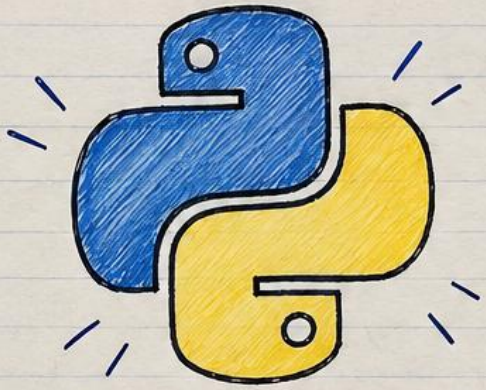
@abhi_techhub

Python Internals & Interview Questions

Code
Learn
Grow







- [x] Core Concepts
- [x] Internal Mechanisms
- [x] Memory Management
- [x] Python Object Model
- [x] Built-in Types
- [x] Functions, Generators
- [x] Decorators
- [x] Concurrency & GIL
- [x] Tricky Interview Questions
- [x] Practice Questions

From Core Concepts to CPython Internals

A Handwritten Guide for Developers

Understand Python beyond the syntax !

Source Code (.py)
↓
Compiler
↓
Bytecode (.pyc)
↓
Python Virtual Machine (PVM)
↓
OS / Hardware

Know How Python really works under the hood !

Why this Handbook ?

- Clear & concise notes
- Visual explanations
- Real code examples
- Covers interview questions
- Helps in building deeper understanding
- Useful for SDE interviews

@abhi_techhub

Same Concepts Bigger Opportunities !

1. Introduction to Python Internals

What are Python Internals ?

- Python internals refers to how Python works under the hood.
- It includes the implementation details of the Python interpreter (mainly CPython).
- Understanding internals helps us write better, efficient code and answer interview questions.
- It also helps in debugging, performance tuning and contributing to Python.

Why is it important ?



1. Helps in writing efficient code
2. Useful for technical interviews
3. Helps in debugging tricky issues
4. Better understanding of memory, performance and concurrency
5. Useful if you want to contribute to Python (open source)

Know how things work under the hood !

Different Python Implementations

Implementation	Language	Description
CPython	C	Default and most widely used implementation.
PyPy	Python	Fast implementation using JIT (Just-In-Time) compilation.
Jython	Java	Runs on JVM, can use Java libraries.
IronPython	C#	Runs on .NET framework.
MicroPython	C	Lightweight implementation for embedded systems (microcontrollers).

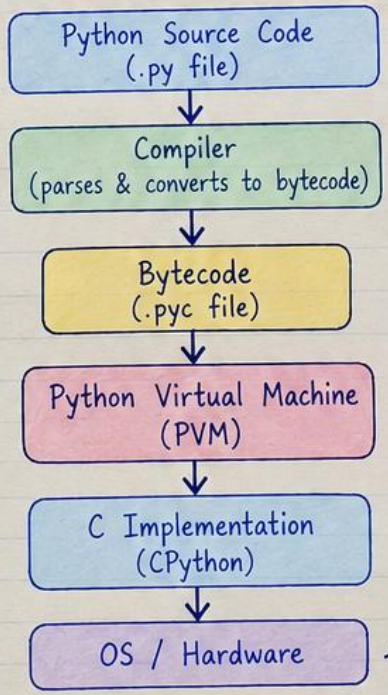
Focus of these Notes

- In this handbook, we will mainly focus on CPython internals, as it is the default implementation and most relevant for interviews.

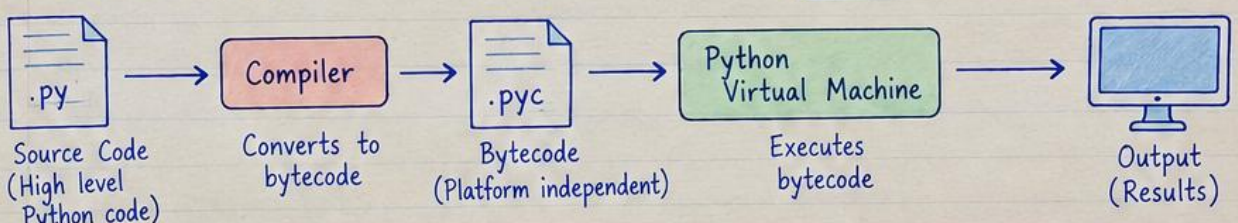
How Python Works ? (Overview)

1. You write Python code in a .py file.
2. The source code is compiled to bytecode (.pyc).
3. The Python Virtual Machine (PVM) executes the bytecode.
4. PVM is implemented in C (in CPython).
5. PVM interacts with the operating system and underlying hardware.

High Level Architecture (CPython)



This is what we will study in detail in next pages !



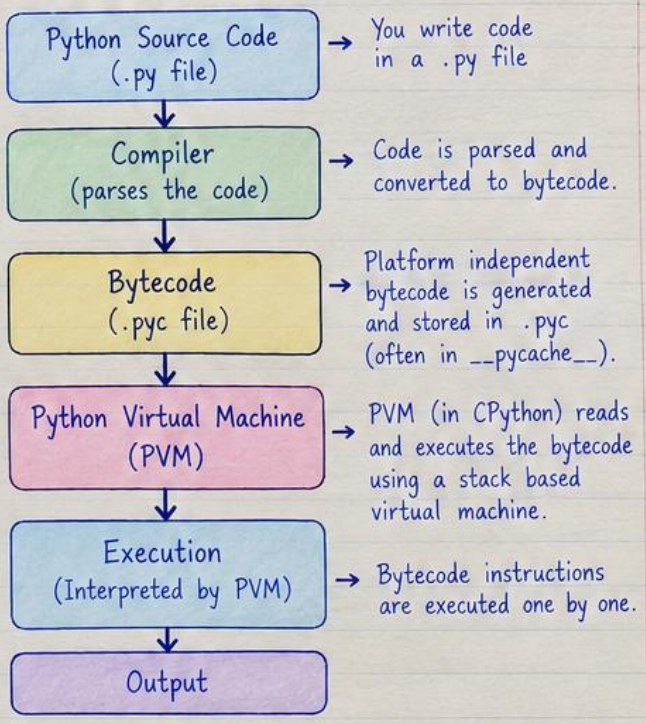
Key Takeaways

- Python is an interpreted, high-level, general purpose programming language.
- CPython is the most popular implementation (written in C).
- Python code is compiled to bytecode, which is executed by PVM.
- Understanding internals helps in writing better, performant and bug-free code.

2. How Python Executes Code

Source Code → Bytecode → Execution (Step by Step)

1 High Level Flow



2 Example Code

```
def add(a, b):
    c = a + b
    return c

result = add(2, 3)
print(result)
```

This is our high level Python code.

3 Compile to Bytecode

- We can inspect the bytecode using the dis module.

```
import dis

def add(a, b):
    c = a + b
    return c

dis.dis(add)
```

dis.dis() shows the bytecode instructions generated for the function.

4 Output (Bytecode)

```
  2   0  RESUME        0
      2  LOAD_FAST     0 (a)
      4  LOAD_FAST     1 (b)
      6  BINARY_OP     0 (+)
     10  STORE_FAST    2 (c)
     12  LOAD_FAST     2 (c)
     14  RETURN_VALUE
```

Common Bytecode Instructions

- LOAD_FAST – Load local variable onto stack
- LOAD_CONST – Load constant value
- BINARY_OP – Perform binary operation (+, -, *)
- STORE_FAST – Store value in local variable
- RETURN_VALUE – Return from function
- RESUME – Start/resume execution (Python 3.11+)

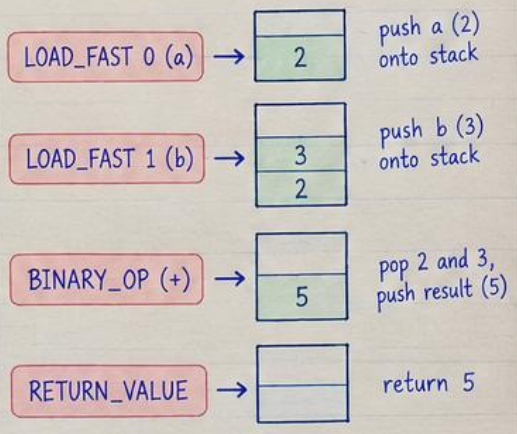
5 How Execution Happens ? (Inside PVM)

- PVM uses a stack based architecture.
- Each bytecode instruction is executed sequentially.
- Operands are pushed to a stack.
- Operations (like +) pop values from the stack, compute and push the result back.
- The PVM is implemented in C (in CPython) and interacts with the underlying operating system.

Key Takeaways

- Python code is compiled to bytecode, not machine code.
- Bytecode is executed by the Python Virtual Machine (PVM).
- We can inspect bytecode using the dis module.
- PVM is stack based and implemented in C (for CPython).
- Bytecode makes Python platform independent.

6 Stack Execution Example



@abhi_techhub

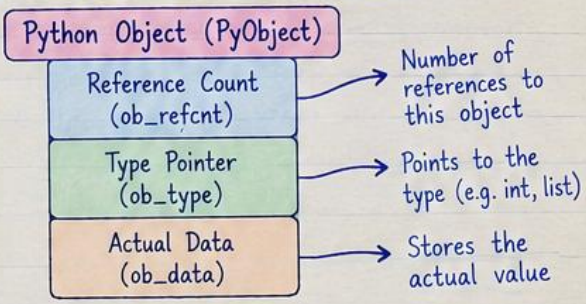
3. Python Memory Management

Reference Counting, Garbage Collection & pymalloc

1 How is memory managed in Python ?

- Python manages memory automatically.
- Objects are allocated on the heap.
- Memory is managed using a combination of:
  1. Reference Counting (immediate cleanup)
  2. Garbage Collection (cyclic references)
  3. pymalloc (specialized memory allocator)

2 Object in Memory (CPython)



3 Reference Counting

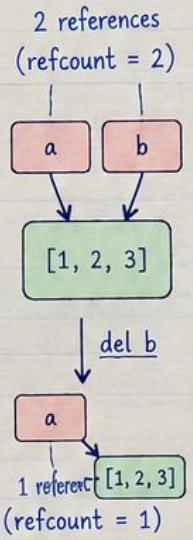
- Each object has a reference count.
- When reference count becomes 0, memory is freed immediately.
- Implemented in CPython.

```
import sys

a = [1, 2, 3]
print(sys.getrefcount(a))   # 2

b = a
print(sys.getrefcount(a))   # 3

del b
print(sys.getrefcount(a))   # 2
```



4 Garbage Collection (Cyclic References)

- Reference counting cannot handle cyclic references.
- Python uses a cyclic garbage collector (gc module) to detect and collect such objects.
- It uses a generational garbage collection approach (0, 1, 2).

Example: Cyclic Reference

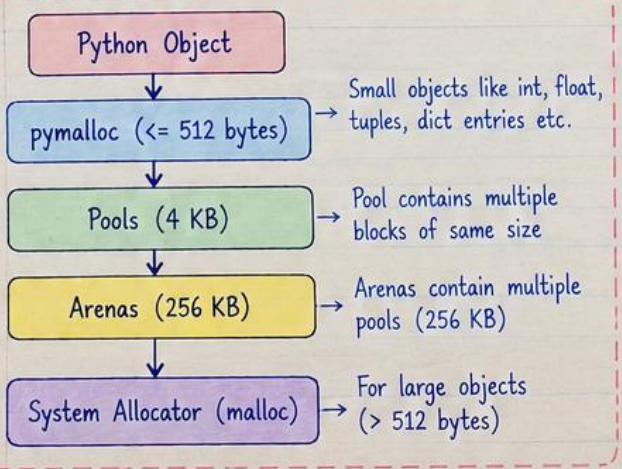
```
import gc
class Node:
  def __init__(self):
    self.ref = None

a = Node()
b = Node()
a.ref = b
b.ref = a
del a, b
gc.collect()  # Force garbage collection
```

Here a and b reference each other. Reference count never becomes 0, so garbage collector will clean this up.

5 pymalloc - Python's Memory Allocator

- CPython uses a custom memory allocator called pymalloc for small objects (<= 512 bytes).
- It maintains pools and arenas to reduce memory fragmentation and improve performance.
- For larger objects, it uses the system malloc().



6 Generational GC

- Objects are divided into 3 generations:
  0. Generation 0 - New objects (collected most frequently)
  1. Generation 1 - Survive longer
  2. Generation 2 - Long lived objects (collected less frequently)
- If an object survives a collection, it moves to the next generation.

7 Useful gc module functions

```
import gc
gc.get_count()      # number of allocations
gc.get_threshold()  # collection thresholds
gc.collect()        # force garbage collection
gc.disable()        # disable GC
gc.enable()         # enable GC
```

Key Takeaways

- Python uses reference counting + cyclic GC.
- pymalloc handles small object memory allocation.
- Cyclic references are cleaned by garbage collector.
- Memory management is implementation specific (CPython).

4. Python Object Model

Everything in Python is an Object

In Python, every value is an object !

1 What is an Object ?

- An object is a real entity in memory with identity, type and value.
- Every item in Python (int, float, list, function, class, module, etc.) is an object.
- Objects have attributes and methods.
- In CPython, every object is a PyObject (written in C).

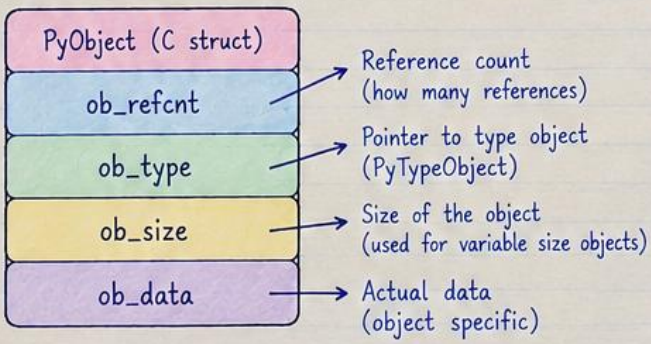
2 Object Identity, Type and Value

```
a = 10
b = 10
print(a == b)     # True (value is same)
print(a is b)     # True (small integers are cached)
print(id(a))
print(id(b))      # same id (in CPython)
```

Identity vs Equality

- == checks the value of objects.
- is checks whether two variables refer to the same object (same memory location).

3 Python Object Structure (In CPython)



4 Type and Class

- type() returns the type (class) of an object.
- In Python, classes are also objects (their type is 'type').

```
print(type(10))           # <class 'int'>
print(type([]))           # <class 'list'>
print(type(type))         # <class 'type'>
print(isisntance(10, int))  # True
```

type is a metaclass, it is the class of all classes in Python.

5 Common Built-in Types (All are Objects)

Object	Type	Example	Notes
Integer	int	10	Immutable
Float	float	3.14	Immutable
String	str	"hello"	Immutable
List	list	[1, 2, 3]	Mutable
Tuple	tuple	(1, 2, 3)	Immutable
Dict	dict	{'a': 1}	Mutable
Set	set	{1, 2, 3}	Mutable
Function	function	def add(): ...	Object
Class	type	class A: ...	Object
Module	module	import sys	Object

6 Attributes and Methods

- Objects have attributes (data) and methods (functions).
- We can access them using dot notation.

```
s = "Python"
print(s.upper())    # method of string object
print(s.__class__)  # <class 'str'>
print(s.__len__())  # 6
print(dir(s))       # list of all attributes
                    # and methods
```

7 Custom Class Example (User Defined Object)

```
class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def greet(self):
        return f"Hi, I am {self.name}"

p = Person("Abhi", 25)
print(type(p))      # <class '__main__.Person'>
print(p.name)       # Abhi
print(p.greet())    # Hi, I am Abhi
```

p is an object of class Person. It has attributes name, age and a method greet().

8 Key Takeaways

- Everything in Python is an object.
- Each object has an identity (id), type and value.
- Objects are instances of a class.
- Classes are also objects (type is 'type').
- Use type(), isinstance() and dir() to explore objects.
- In CPython, every object is a PyObject struct.
- Immutable objects (e.g., int, str, tuple) cannot be changed, mutable objects (e.g., list, dict) can be modified.

@abhi_techhub

5. Mutable vs Immutable Objects

Understanding mutability, references and common interview traps

1 What does Mutable and Immutable mean ?

- Mutable objects can be modified after creation (their state can change).
- Immutable objects cannot be modified after creation (any operation creates a new object).
- In Python, variables are references to objects, not the actual objects.

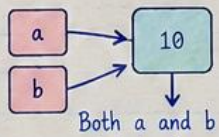
2 Examples

Immutable Objects	Mutable Objects
• int • float • str • bool • tuple • frozenset • bytes • NoneType	• list • dict • set • bytearray • class instances • most built-in custom objects

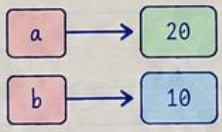
3 Immutable Example (int, str, tuple)

```
a = 10
b = a
print(id(a))  # same id
print(id(b))

a = 20        # new object created
print(id(a))  # different id
print(id(b))  # b still points to 10
```



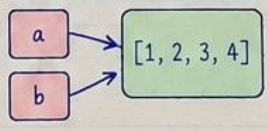
Both a and b point to the same immutable object



a now points to new object

4 Mutable Example (list, dict, set)

```
a = [1, 2, 3]
b = a
print(id(a))      # same id
print(id(b))
a.append(4)
print(a)          # [1, 2, 3, 4]
print(b)          # [1, 2, 3, 4]
print(id(a))      # same id
print(id(b))
```



Both a and b refer to the same list object. Changes are visible to both.

5 Mutability with Function Arguments

```
def modify(x):
    x.append(4)

l = [1, 2, 3]
modify(l)
print(l)      # [1, 2, 3, 4]
```

Explanation:

- The list is passed by reference (object reference).
- The function modifies the same list object.

6 Immutable with Function Arguments

```
def modify(x):
    x = x + 10
    print("Inside:", id(x))

a = 10
print("Before:", id(a))
modify(a)
print("After:", id(a))
```

Explanation:

- Integers are immutable.
- A new object is created inside the function.
- Original object remains unchanged.

7 Common Interview Traps

1 Mutable default argument

```
def add(x, lst=[]):
    lst.append(x)
    return lst

print(add(1))     # [1]
print(add(2))     # [1, 2] (unexpected!)
```

Default list is created only once, so the same list is used across calls. Use None instead !

2 List multiplication creates shallow copy

```
a = [[0] * 3] * 2
a[0][0] = 1
print(a)   # [[1, 0, 0], [1, 0, 0]]
```

Both inner lists refer to the same list object.

3 is vs ==

```
a = [1, 2, 3]
b = [1, 2, 3]
print(a == b)    # True (values are same)
print(a is b)    # False (different objects)
```

4 Tuple with mutable objects

```
t = (1, 2, [3, 4])
t[2].append(5)
print(t)   # (1, 2, [3, 4, 5])
```

Tuple is immutable but it can contain mutable objects. We can modify the mutable object inside.

8 Key Takeaways

- Mutable objects can be changed, immutable cannot.
- Variables are references to objects.
- Use id() to check if two variables point to the same object.
- Be careful with mutable default arguments.
- Tuple is immutable, but its elements can be mutable.

@abhi_techhub

6. Namespaces & Scope

The LEGB Rule, Closures, global and nonlocal

A namespace is a mapping of names to objects.

1 What is a Namespace ?

- A namespace is a dictionary-like mapping of names (identifiers) to objects.
- It helps in avoiding name conflicts.
- Every module, function, class and block has its own namespace.
- We can inspect the current namespace using the globals() and locals() functions.

```
x = 10
y = 20
print(globals()["x"])   # 10
print(locals()["y"])    # 20
```

2 The LEGB Rule (Scope Resolution)

- When Python looks for a variable, it searches in the following order:

L	Local	→ inside current function
E	Enclosing	→ in enclosing functions (for nested functions)
G	Global	→ in the current module
B	Built-in	→ in Python built-in namespace (e.g. len, print, list)

Search order ↑

3 Local Scope

- Variables defined inside a function are local to that function.
- They are not accessible outside.

```
def func():
    x = 10
    print(x)    # 10

func()
print(x)        # NameError
```

4 Enclosing Scope (Closure)

- Nested functions can access variables from their enclosing scope.
- This is how closures work.

```
def outer():
    x = 10
    def inner():
        print(x)  # 10 (from enclosing)
    return inner

f = outer()
f()
```

5 Global Scope

- Variables defined at the module level are global.
- Use global keyword to modify a global variable inside a function.

```
x = 5

def update():
    global x
    x = 20

update()
print(x)    # 20
```

6 Built-in Scope

- If a name is not found in local, enclosing or global scope, Python looks in the built-in namespace (e.g. len, print, sum).

```
print(len([1,2,3]))   # 3
print(sum([1,2,3]))   # 6
```

7 nonlocal Keyword

- Used to modify a variable in the enclosing scope (but not global).
- It allows inner functions to update the value of variables from outer functions.

```
def outer():
    x = 10
    def inner():
        nonlocal x
        x = x + 1
        print(x)
    inner()
    print(x)   # 11
outer()
```

Without nonlocal, assigning to x inside inner() will create a new local variable, leading to UnboundLocalError if we try to access x before assignment.

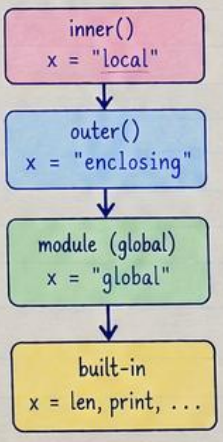
8 Name Resolution Example

Let's see how Python resolves a variable:

```
x = "global"
def outer():
    x = "enclosing"
    def inner():
        x = "local"
        print(x)  # local

    inner()
    print(x)   # enclosing

outer()
print(x)       # global
```



9 Key Takeaways

- Namespaces map names to objects.
- Python follows the LEGB rule for variable lookup.
- Use global to modify a global variable inside a function.
- Use nonlocal to modify a variable in the enclosing scope.
- Each module, function and class has its own namespace.
- Built-in namespace contains functions like len, print, etc.
- Closures allow inner functions to access variables from enclosing scope.
- Understanding scope helps in avoiding name conflicts and debugging issues.

@abhi_techhub

7. Python Functions Internals

First-Class Objects, Closures, Arguments & Function Attributes

1 Functions are First-Class Objects

- In Python, functions are objects.
- They can be assigned to variables, passed as arguments, returned from other functions and stored in data structures.
- A function is an instance of class 'function'.

```
def add(a, b):
    return a + b

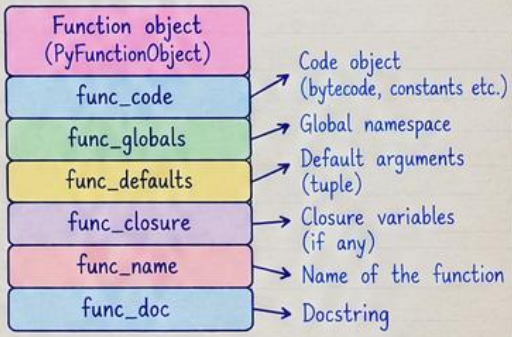
print(type(add))    # <class 'function'>
f = add             # assign to variable
print(f(2, 3))      # 5
```

Functions can be:

- Assigned
- Passed as argument
- Returned from function
- Stored in list/dict
- Have attributes
- Have a __call__()

2 Function Object Internals

- In CPython, a function is represented by PyFunctionObject (in funcobject.c).
- It contains a code object, globals, defaults, closure and other attributes.



3 Code Object (func.__code__)

- Every function has a code object which contains bytecode and metadata about the function.
- We can inspect it using __code__ attribute.

```
def multiply(a, b):
    return a * b

print(multiply.__code__)
print(multiply.__code__.co_name)     # 'multiply'
print(multiply.__code__.co_argcount) # 2
print(multiply.__code__.co_consts)   # (None,)
print(multiply.__code__.co_varnames) # ('a', 'b')
print(multiply.__code__.co_filename) # file name
print(multiply.__code__.co_firstlineno) # line number
```

Important attributes:

- co_name
- co_argcount
- co_varnames
- co_consts
- co_filename
- co_firstlineno
- co_code (bytecode)
- co_freevars (for closures)

4 Arguments in Python Functions

- Python functions support many types of arguments.

```
def example(a, b, c=10, *args, d=20, **kwargs):
    print(a, b, c)
    print(args)
    print(d)
    print(kwargs)
```

Function call:

```
example(1, 2, 3, 4, 5, d=100, x=9, y=8)
```

Output:

```
1 2 3
(4, 5)
100
{'x': 9, 'y': 8}
```

Types of arguments:

- Positional arguments
- Default arguments
- Variable length (*args)
- Keyword-only arguments
- Arbitrary keyword (**kwargs)

5 Default Arguments are Evaluated Once

- Default argument values are evaluated only once at function definition time.

```
def append_item(item, lst=[]):
    lst.append(item)
    return lst

print(append_item(1))   # [1]
print(append_item(2))   # [1, 2] (unexpected!)
```

Explanation:

- The same list object is used across calls.
- To avoid this, use None and create a new object inside the function.

6 Closures and Free Variables

- A closure is a function that remembers variables from its enclosing scope, even after the outer function has finished executing.

```
def outer(x):
    def inner(y):
        return x + y
    return inner
f = outer(10)
print(f(5))     # 15
```

Closure internals:

- inner function keeps a reference to x.
- We can access closure variables using __closure__.
- Each cell contains the value of a free variable.

7 Function Attributes and Utilities

- Functions have useful built-in attributes.

```
def greet(name):
    """This is a docstring"""
    return f"Hi {name}"

print(greet.__name__)     # greet
print(greet.__doc__)      # This is a docstring
print(greet.__defaults__) # None
print(greet.__code__)     # code object
```

We can also modify attributes:

```
def hello():
    pass

hello.__name__ = "new_name"
hello.custom_attr = "value"
print(hello.__name__) # new_name
print(hello.custom_attr) # value
```

8 Key Takeaways

- Functions are first-class objects in Python.
- Each function has a code object (__code__).
- Default arguments are evaluated once.
- Functions can have variable length arguments (*args, **kwargs).
- Closures remember variables from enclosing scope.
- We can inspect and modify function attributes.
- Functions are instances of class 'function' (PyFunctionObject in CPython).

8. Iterators & Generators

Lazy Evaluation, yield and the Iterator Protocol

1 Iterable vs Iterator

- Iterable: An object that can return an iterator using __iter__(). We can loop over it.
- Iterator: An object that represents a stream of data. It has __iter__() and __next__() methods and returns next value using __next__().

Feature	Iterable	Iterator
__iter__()	Yes	Yes (returns self)
__next__()	No	Yes
Can be looped	Yes	Yes
Example	list, tuple, dict, str, range	iter(list), generator objects, file objects

2 Iterator Protocol

- An iterator is an object that implements __iter__() and __next__().
- __iter__() returns the iterator object (usually self).
- __next__() returns the next value. When no more elements are left, it raises StopIteration.

```
class Counter:
    def __init__(self, n):
        self.n = n
        self.current = 0

    def __iter__(self):
        return self

    def __next__(self):
        if self.current >= self.n:
            raise StopIteration
        val = self.current
        self.current += 1
        return val
```

Using the iterator

```
for i in Counter(3):
    print(i)

# Output:
# 0
# 1
# 2
```

3 Generators

- Generators are a simple way to create iterators.
- They use yield instead of return.
- They are memory efficient (lazy evaluation).

```
def square_numbers(n):
    for i in range(n):
        yield i * i

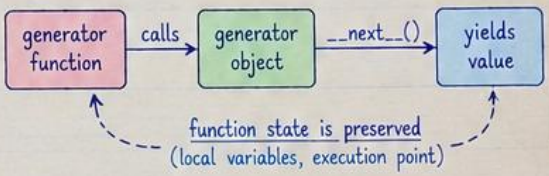
gen = square_numbers(5)
print(gen)      # <generator object ...>
for val in gen:
    print(val)  # 0 1 4 9 16
```

Why use generators ?

- Do not store all values in memory.
- Values are generated on the fly.
- Useful for large data, streams and pipelines.

4 Generator Internals

- When a function contains yield, it becomes a generator function.
- Calling it returns a generator object.
- The function state (local variables, instruction pointer) is preserved between yields.



5 Generator Expressions

- A compact way to create generators.
- Similar to list comprehension but uses () instead of [].

```
squares = (x * x for x in range(5))
print(type(squares))   # <class 'generator'>
for x in squares:
    print(x)           # 0 1 4 9 16
```

List vs Generator

List: [x*x for x in range(5)]
→ Creates all values in memory.

Generator: (x*x for x in range(5))
→ Creates values lazily.

6 Use Cases

- Reading large files line by line.
- Processing large datasets.
- Data pipelines (e.g. logs, streams).
- Infinite sequences (using generators).

```
# Infinite generator (numbers)
def counter():
    n = 0
    while True:
        yield n
        n += 1

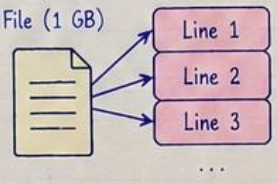
for i, val in zip(range(5), counter()):
    print(val)   # 0 1 2 3 4
```

7 Real World Example - Reading a Large File

- Using a generator to read a file line by line (memory efficient).

```
def read_file(file_path):
    with open(file_path, 'r') as f:
        for line in f:
            yield line.strip()

for line in read_file('big_file.txt'):
    print(line)
    # process line by line
```



Read one line at a time
(not entire file in memory).

8 Key Takeaways

- Iterable can return an iterator using __iter__().
- Iterator implements __iter__() and __next__().
- Generators use yield and are lazy (memory efficient).
- Generator expression is a concise way to create generators.
- Generators maintain their state between yields.
- Useful for large files, streams, and infinite sequences.
- Use lists when all data fits in memory, otherwise use generators.

10. Python Decorators

Wrappers, Closures, functools.wraps and Real-World Examples

A decorator adds extra functionality to a function without modifying its code.

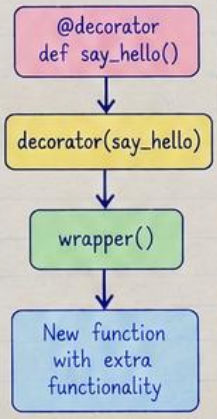
1 What is a Decorator ?

- A decorator is a function that takes another function as input, adds some functionality and returns a new function.
- It uses closures to remember the original function.
- Does not modify the original function code.
- Commonly used for logging, authentication, caching, timing, retry mechanism, etc.

```
def decorator(func):
    def wrapper(*args, **kwargs):
        print("Before function call")
        result = func(*args, **kwargs)
        print("After function call")
        return result
    return wrapper

@decorator
def say_hello():
    print("Hello!")

say_hello()
```



2 How Decorators Work (Step by Step)

1. The decorated function is passed to the decorator.
2. Decorator returns a new function (wrapper).
3. The wrapper function adds extra behavior and calls the original function.
4. The name now refers to the wrapper function.

```
@decorator
def add(a, b):
    return a + b
```

is equivalent to

```
add = decorator(add)
```

Now 'add' refers to the wrapper function.

3 Decorator with Arguments

- We can also pass arguments to a decorator using an extra layer of function.

```
def repeat(times):
    def decorator(func):
        def wrapper(*args, **kwargs):
            for _ in range(times):
                result = func(*args, **kwargs)
            return result
        return wrapper
    return decorator

@repeat(3)
def greet():
    print("Hello!")

greet()
```

4 Preserving Metadata with functools.wraps

- When we use a decorator, the wrapper function replaces the original function, so its metadata (__name__, __doc__, etc.) is lost.
- We use functools.wraps to preserve the original function's metadata.

```
from functools import wraps
def log_function(func):
    @wraps(func)
    def wrapper(*args, **kwargs):
        print(f"Calling {func.__name__}")
        return func(*args, **kwargs)
    return wrapper

@log_function
def add(a, b):
    """Adds two numbers"""
    return a + b

print(add.__name__)  # add
print(add.__doc__)   # Adds two numbers
```

5 Common Use Cases

Logging	Log function calls and arguments.
Authentication	Check user authentication.
Timing	Measure function execution time.
Caching	Cache function results (memoization).
Retry	Retry on failures (e.g., API calls).
Access Control	Restrict access to functions.
Monitoring	Track usage, metrics, etc.

6 Real World Examples

1. Timing Decorator

```
import time
from functools import wraps
def timer(func):
    @wraps(func)
    def wrapper(*args, **kwargs):
        start = time.time()
        result = func(*args, **kwargs)
        end = time.time()
        print(f"Time: {end - start:.4f}s")
        return result
    return wrapper

@timer
def slow_function():
    time.sleep(2)
    print("Done")

slow_function()
```

2. Cache Decorator (Simple)

```
def cache(func):
    cache_dict = {}
    @wraps(func)
    def wrapper(*args):
        if args in cache_dict:
            print("Returning cached result")
            return cache_dict[args]
        result = func(*args)
        cache_dict[args] = result
        return result
    return wrapper

@cache
def fibonacci(n):
    if n <= 1:
        return n
    return fibonacci(n-1) + fibonacci(n-2)

print(fibonacci(10))  # 55
```

3. Authentication Decorator

```
def require_login(func):
    @wraps(func)
    def wrapper(*args, **kwargs):
        user = get_current_user()  # dummy
        if not user:
            print("Please login first")
            return None
        return func(*args, **kwargs)
    return wrapper
```

7 Key Takeaways

- Decorators are higher-order functions.
- They use closures to remember the original function.
- Use functools.wraps to preserve metadata.
- Decorators can accept arguments.
- Commonly used for logging, timing, caching, authentication, etc.
- Decorators return a new function, they don't modify the original.

@abhi_techhub

11. Python Concurrency & the GIL

Threads vs Processes, CPU-Bound vs I/O-Bound Tasks

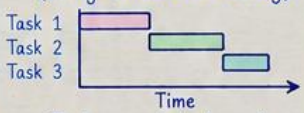
1 What is Concurrency ?

- Concurrency means dealing with multiple tasks at the same time.
- It helps in better resource utilization and improves performance.
- In Python, we achieve concurrency using Threads, Processes, Asyncio, etc.
- It is not the same as parallelism, but concurrency can lead to parallelism (in multi-core systems).

2 Concurrency vs Parallelism

Concurrency

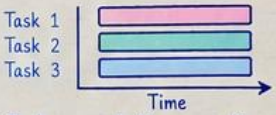
- Multiple tasks make progress (not necessarily at the same time).
- Deals with managing multiple tasks.
- Can run on a single core (using context switching).



Tasks are interleaved (concurrent)

Parallelism

- Multiple tasks run simultaneously.
- Actually executes at the same time.
- Requires multiple CPU cores or multiple machines.



Tasks run at the same time (in parallel)

3 The Global Interpreter Lock (GIL)

- In CPython, only one thread can execute Python bytecode at a time due to GIL.
- GIL prevents multiple native threads from executing Python bytecode simultaneously (even on multi-core CPUs).
- It simplifies memory management in CPython but limits CPU-bound parallelism.



4 CPU-Bound vs I/O-Bound Tasks

CPU-Bound Tasks

- Tasks that require heavy computation.
- Examples: image processing, ML training, large calculations.
- Due to GIL, multithreading won't give true parallelism in CPU-bound tasks.
- Use multiprocessing for CPU-bound tasks.

Example: Calculating factorial

```
from time import time

def factorial(n):
    if n <= 1:
        return 1
    return n * factorial(n-1)

start = time()
print(factorial(10000))
print("Time:", time() - start)
```

I/O-Bound Tasks

- Tasks that spend more time waiting for I/O operations (network, file, database, etc).
- Examples: API calls, reading files, database queries.
- Multithreading or asyncio works well for I/O-bound tasks (because while one thread is waiting, another can run).

Example: API calls (I/O bound)

```
import time
import requests

def fetch_data(url):
    response = requests.get(url)
    return response.status_code

start = time.time()
print(fetch_datg("https://example.com"))
print("Time:", time.time() - start)
```

5 Multithreading in Python (threading module)

- Useful for I/O-bound tasks.
- Threads share the same memory space.
- Due to GIL, not effective for CPU-bound tasks.

```
import threading
import time

def worker(n):
    print(f"Thread {n} starting")
    time.sleep(2)
    print(f"Thread {n} finished")

threads = []
for i in range(3):
    t = threading.Thread(target=worker, args=(i,))
    t.start()
    threads.append(t)

for t in threads:
    t.join()
print("All threads completed")
```

Output (approx):

```
Thread 0 starting
Thread 1 starting
Thread 2 starting
Thread 0 finished
Thread 1 finished
Thread 2 finished
All threads completed
```

6 Multiprocessing in Python (multiprocessing module)

- Useful for CPU-bound tasks.
- Each process has its own Python interpreter and memory space.
- Can achieve true parallelism (bypasses GIL).

```
from multiprocessing import Process
import time
def worker(n):
    print(f"Process {n} starting")
    time.sleep(2)
    print(f"Process {n} finished")

processes = []
for i in range(3):
    p = Process(target=worker, args=(i,))
    p.start()
    processes.append(p)

for p in processes:
    p.join()
print("All processes completed")
```

Output (approx):

```
Process 0 starting
Process 1 starting
Process 2 starting
Process 0 finished
Process 1 finished
Process 2 finished
All processes completed
```

7 Asynchronous Programming (asyncio)

- Used for I/O-bound tasks with single thread (event loop).
- Useful for high-concurrency applications (APIs, web servers).

```
import asyncio

async def fetch_data(n):
    print(f"Task {n} started")
    await asyncio.sleep(2)
    print(f"Task {n} finished")

async def main():
    tasks = [fetch_data(i) for i in range(3)]
    await asyncio.gather(*tasks)

asyncio.run(main())
```

Output (approx):

```
Task 0 started
Task 1 started
Task 2 started
Task 0 finished
Task 1 finished
Task 2 finished
```

8 Key Takeaways

- GIL allows only one thread to execute Python bytecode at a time (in CPython).
- Use threading or asyncio for I/O-bound tasks.
- Use multiprocessing for CPU-bound tasks.
- Threads share memory, processes have separate memory.
- Asyncio uses a single thread with an event loop.
- Concurrency improves performance, but parallelism achieves true simultaneous execution.
- Choose the right concurrency model based on the type of task.

@abhi_techhub

12. Python Built-in Types & Internals

Lists, Dictionaries, Sets, Hashing & Time Complexity

1 Built-in Types in Python

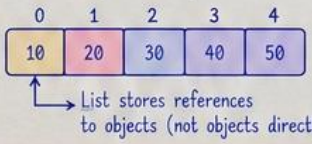
- Python has several built-in types like int, float, str, list, tuple, dict, set, bool, None, etc.
- Each type is an object with its own internal representation in CPython.
- Built-in types are implemented in C for better performance.

Some common built-in types:

```
x = 10              type(x)   # <class 'int'>
s = "hello"         type(s)   # <class 'str'>
lst = [1, 2, 3]     type(lst) # <class 'list'>
t = (1, 2, 3)       type(t)   # <class 'tuple'>
d = {'a': 1}        type(d)   # <class 'dict'>
st = {1, 2, 3}      type(st)  # <class 'set'>
b = True            type(b)   # <class 'bool'>
n = None            type(n)   # <class 'NoneType'>
```

2 List Internals (Dynamic Array)

- List is a dynamically resizable array of pointers to objects.
- Implemented in C as a dynamic array (PyListObject).
- When capacity is full, Python allocates more memory (usually 1.5x the size).



Example:

```
lst = [1, 2, 3]
lst.append(4)
print(lst)    # [1, 2, 3, 4]
print(id(lst[0]))   # memory address
                    of object 1
```

3 List Time Complexity

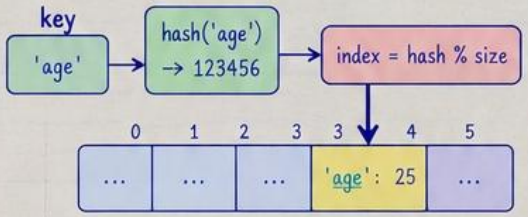
Operation	Time Complexity
Access (lst[i])	O(1)
Append (lst.append)	Amortized O(1)
Insert (lst.insert)	O(n)
Delete (del lst[i])	O(n)
Search (x in lst)	O(n)
Get length (len(lst))	O(1)

Why append is O(1) ?

- Python over-allocates memory (e.g. 1.5x) to avoid frequent resizing.
- So most of the time append is O(1) (amortized).
- But in worst case (when resizing is neeeded) it can be O(n).

4 Dictionary Internals (Hash Table)

- Dictionary stores key-value pairs using a hash table.
- Keys are hashed to find the index.
- Uses open addressing with perturbation for collision resolution (like CPython's dict).
- Keys must be immutable and hashable (e.g. str, int, tuple).



5 Dictionary Time Complexity

Operation	Time Complexity
Insert (d[key] = val)	O(1)
Access (d[key])	O(1)
Delete (del d[key])	O(1)
Search (key in d)	O(n)
Get keys/values	O(n)
Iteration	O(n)

Example:

```
d = {'name': 'abhi', 'age': 25}
print(d['name'])      # abhi
d['city'] = 'Gurgaon' # insert
print(d)  # {'name': 'abhi', 'age': 25,
            'city': 'Gurgaon'}
```

6 Set Internals (Hash Table)

- Set stores unique elements using a hash table (similar to dict, but only keys).
- Elements must be hashable.
- Useful for fast membership testing and removing duplicates.

Example:

```
s = {1, 2, 3, 2}
print(s)    # {1, 2, 3}
s.add(4)
s.remove(2)
print(s)    # {1, 3, 4}
```

7 Set Time Complexity

Operation	Time Complexity
Add (s.add)	O(1)
Remove (s.remove)	O(1)
Search (x in s)	O(1)
Union (s1 | s2)	O(n)
Intersection (s1 & s2)	O(min(n, m))
Iteration	O(n)

8 Hashing in Python

- Hashing converts an object into a fixed-size integer (hash value).
- Used in dict and set for fast lookups.
- Hash value determines the index in the table.
- If two objects are equal, they must have the same hash value.

Example:

```
print(hash('abhi'))     # e.g. 12345678
print(hash(10))         # 10
print(hash((1, 2)))     # hashable (tuple)
# print(hash([1,2]))    # TypeError (list is mutable)
```

Hashable types:
int, float, str, tuple, frozenset

Not hashable:
list, dict, set (mutable types)

9 Tuple Internals

- Tuple is an immutable sequence.
- Implemented as a fixed-size array of pointers.
- Faster than lists and can be used as dict keys (if elements are hashable).

Example:

```
t = (1, 2, 3)
print(t[0])   # 1
# t[0] = 10   # TypeError (immutable)
```

10 Numeric & String Internals (Brief)

- int: Arbitrary precision integers (no size limit).
- float: Implemented using C double (64-bit).
- str: Immutable sequence of Unicode characters (PEP 393 - flexible string representation).

Example:

```
x = 10 ** 100  # Python handles big integers
s = 'hello'
print(len(s))  # 5
print(s[0])    # h
```

11 Key Takeaways

- Lists are dynamic arrays (fast access).
- Dict and set use hash tables for O(1) lookup.
- Keys in dict and elements in set must be hashable and immutable.
- Tuples are immutable and can be used as dict keys.
- Append in list is amortized O(1).
- Dict preserves insertion order (Python 3.7+).
- Use appropriate data structure based on use case (list for ordered data, set for uniqueness, dict for key-value mapping).

13. Tricky Python Interview Questions

Predict the Output, Explain Why, and Understand the Internals

1 Mutable default arguments

```
def add_item(item, lst=[]):
    lst.append(item)
    return lst

print(add_item(1))
print(add_item(2))
print(add_item(3))
```

Output:

```
[1]
[1, 2]
[1, 2, 3]
```

Explanation:

- Default arguments are evaluated only once at function definition time.
- Same list is used across calls.

2 List multiplication (shallow copy)

```
matrix = [[0] * 3] * 3
matrix[0][0] = 1
print(matrix)
```

Output:

```
[[1, 0, 0],
 [1, 0, 0],
 [1, 0, 0]]
```

Explanation:

- [[0] * 3] creates one inner list [0,0,0].
- * 3 copies the reference, not a new list (shallow copy).
- All rows refer to the same inner list.

3 is vs ==

```
a = [1, 2, 3]
b = [1, 2, 3]
print(a == b)
print(a is b)
```

Output:

```
True
False
```

Explanation:

- == compares values.
- is compares object identity (memory address).
- a and b are different objects.

4 What is printed ? (closures)

```
funcs = []
for i in range(3):
    def f():
        return i
    funcs.append(f)

for f in funcs:
    print(f())
```

Output:

```
2
2
2
```

Explanation:

- All functions capture the same variable i (by reference).
- After loop ends, i = 2, so all return 2.

5 Fixing the closure issue

```
funcs = []
for i in range(3):
    def f(i=i):
        return i
    funcs.append(f)

for f in funcs:
    print(f())
```

Output:

```
0
1
2
```

Explanation:

- Use default argument i=i to capture the current value of i.
- A new default value is created in each iteration.

6 Dictionary key with mutable type

```
d = {}
l = [1, 2, 3]
d[l] = "test"
```

Output:

```
TypeError:
unhashable type: 'list'
```

Explanation:

- Dictionary keys must be hashable (immutable).
- list is mutable and cannot be hashed.

7 set with mutable elements

```
s = set()
s.add([1, 2, 3])
```

Output:

```
TypeError:
unhashable type: 'list'
```

Explanation:

- Set elements must be hashable.
- Use tuple instead: s.add((1, 2, 3))

8 Strange behavior with =, +=

```
a = [1, 2, 3]
b = a
b += [4]
print(a)
print(b)
```

Output:

```
[1, 2, 3, 4]
[1, 2, 3, 4]
```

Explanation:

- b = a copies the reference, not a new list.
- b += [4] modifies the list in-place (same object).
- Both a and b refer to the same list.

9 What is the output ? (ternary + truthy)

```
a = []
result = a and "Yes" or "No"
print(result)
```

Output:

```
No
```

Explanation:

- Empty list is falsy, so a and "Yes" is [].
- [] or "No" returns "No" (first truthy value in or expression).

10 String interning

```
a = "hello"
b = "hello"
c = "he" + "llo"
d = "".join(["he", "llo"])
print(a is b)
print(a is c)
print(a is d)
```

Output:

```
True
True
False
```

Explanation:

- Short strings may be interned by Python.
- "he" + "llo" is optimized at compile time.
- "".join(...) creates a new string object at runtime.

11 Find the output (generator)

```
def gen():
    for i in range(3):
        yield i

it = gen()
print(next(it))
print(list(it))
print(list(it))
```

Output:

```
0
[1, 2]
[]
```

Explanation:

- Generators maintain their state between calls.
- After consuming remaining values, the generator is exhausted.
- Further iteration returns empty list.

12 Tricky with tuples and mutability

```
t = (1, 2, [3, 4])
t[2].append(5)
print(t)
```

Output:

```
(1, 2, [3, 4, 5])
```

Explanation:

- Tuple is immutable, but it can contain mutable objects.
- We modified the inner list, not the tuple structure.

Key Takeaways

- Know the difference between mutable and immutable objects.
- Understand how Python handles object references and memory.
- Default arguments are evaluated once at function definition time.
- Use tuples for immutable collections (hashable), lists for mutable.
- Closures capture variables by reference, not by value.
- Generators are lazy and maintain state.
- Dictionary keys and set elements must be hashable.
- String interning can lead to unexpected behavior with 'is'.
- In-place operations (+=, append, etc.) modify the original object.
- Understand truthy and falsy values in Python.
- Always consider time and space complexity for built-in operations.

@abhi_techhub

14. Python Quick Revision Sheet

Core Concepts, Internals & Interview Cheat Sheet

1 Everything is an Object

- Every value in Python is an object.
- Each object has: identity (id), type and value.
- Use type(), id(), isinstance().

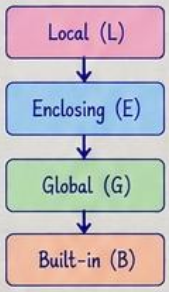
```
x = 10
print(type(x))    # <class 'int">
print(id(x))      # memory address
print(isistance(x), int))  # True
```

2 Immutable vs Mutable

- Immutable: int, float, str, tuple, frozenset (cannot be changed).
- Mutable: list, dict, set (can be changed).
- Mutables are passed by reference.

```
a = [1, 2]
b = a
b.append(3)
print(a)   # [1, 2, 3]
```

3 Namespaces & Scope (LEGB)



- Python searches names in LEGB order.
- Use global to modify global variable.
- Use nonlocal to modify enclosing variable.

```
x = 10
def outer():
    x = 20
    def inner():
        nonlocal x
        x = 30
```

4 Functions Internals

- Functions are first-class objects.
- Have atributes like __name__, __doc__, __code__, __defaults__.
- Functions can be passed, returned and assigned to variables.

```
def add(a, b=1):
    return a + b

print(add.__name__)     # add
print(add.__defaults__)    # (1,)
print(add.__code__.co_argcount)   # 2
```

5 Iterators vs Generators

- Iterable: object with __iter__().
- Iterator: object with __next__().
- Generators use yield and are lazy (memory efficient).

```
def gen(n):
    for i in range(n):
        yield i

g = gen(3)
print(next(g))   # 0
print(next(g))   # 1
```

6 Decorators

- Decorator takes a function and returns a new function with extra behavior.
- Uses closures.
- Use functools.wraps to preserve metadata.

```
from functools import wraps
def logger(func):
    @wraps(func)
    def wrapper(*args, **kwargs):
        print(f"Calling {func.__name__}")
        return func(*args, **kwargs)
    return wrapper
```

7 Concurrency & GIL

- GIL allows only one thread to execute Python bytecode at a time (in CPython).
- Use threading for I/O-bound tasks.
- Use multiprocessing for CPU-bound tasks.
- For async I/O, use asyncio.

Task Type	Use	Example
I/O-bound	Threading / Asyncio	API calls, file I/O
CPU-bound	Multiprocessing	Image processing, ML training

8 Lists, Dicts, Sets Internals

- List: dynamic array, O(1) access, O(1) append (amortized).
- Dict/Set: hash table, O(1) average lookup, keys must be hashable.
- Tuple: immutable sequence.

Operation	List	Dict	Set
Access	O(1)	O(1)	-
Insert	O(1)	O(1)	O(1)
Delete	O(n)	O(1)	O(1)
Search	O(n)	O(1)	O(1)

9 Hashing & Hashable Types

- Hash converts object to fixed-size integer.
- Used in dict and set.
- If two objects are equal, they must have the same hash value.
- Mutable types (list, dict, set) are not hashable.

```
print(hash('abhi'))
s = {1, 2, 3}
# s = {[1, 2]}  # TypeError

d = {('x', 1): 'value'}  # tuple is hashable
```

10 Common Time Complexities

Operation	Time Complexity
List access (lst[])	O(1)
List append	Amortized O(1)
List insert/delete	O(n)
Dict get/set	O(1)
Set add/check	O(1)
Iterate (n elements)	O(n)
List search (x in lst)	O(n)
Set/Dict search	O(1)
Sorting (sorted())	O(n log n)

11 Useful Built-in Functions

Function	Purpose
id()	Memory address (identity)
type()	Get type of object
isinstance()	Check type (supports inheritance)
dir()	List attributes/methods
vars()	Get __dict__ of object
help()	Get documentation
enumerate()	Iterate with index
zip()	Combine iterables
map()/filter()	Functional utilities
sorted()	Return sorted list

12 Common Gotchas

- Mutable default arguments.
- List multiplication creates shallow copy.
- Using 'is' instead of '=='.
- Modifying list while iterating.
- Late binding in closures.
- Mutable objects as dict keys / set elements.
- Generator is exhausted after iteration.
- Strings may be interned (use '==' for comparison).
- Be careful with in-place vs new object operations.
- Floating point precision issues.
- Recursion depth limit (sys.getrecursionlimit()).

13 Useful Modules for Interviews

Module	Use Case
collections	Counter, defaultdict, deque, namedtuple
itertools	Efficient iterators
functools	wraps, lru_cache, partial
threading	Multithreading
multiprocessing	Parallel processing
asyncio	Asynchronous programming
sys	Python runtime information
time	Time-related functions
os	OS interactions
json	JSON parsing

14 Memory Management (Quick)

- Uses reference counting + cycle detection (garbage collector).
- When reference count becomes 0, object is deallocated.
- Circular references are handled by the garbage collector.
- Use del to remove reference.
- Use gc module for manual control if needed.

```
import sys, gc
x = []
print(sys.getrefcount(x))
del x
gc.collect()
```

15 Key Takeaways

- ★ Understand object model and memory.
- ★ Know mutable vs immutable types.
- ★ Be clear about scope, closures and functions internals.
- ★ Understand iterators, generators and decorators.
- ★ Know concurrency, GIL, threading, multiprocessing and asyncio.
- ★ Be familiar with internal working of lists, dicts, sets.
- ★ Understand hashing and equality.
- ★ Practice tricky questions and edge cases.
- ★ Always consider time and space complexity.

15. Practice & Next Steps

Coding Challenges, Revision Checklist, and Learning Plan



1 Coding Challenges (Solve These)

1. Reverse a linked list (use list only, no built-in reverse).
2. Find the first non-repeating character in a string.
3. Implement an LRU Cache (using dict + DLL or OrderedDict).
4. Group anagrams from a list of strings.
5. Find the top K frequent elements.
6. Implement a simple decorator to measure execution time.
7. Write a generator to read a large file line by line.
8. Implement a retry decorator with exponential backoff.
9. Simulate a rate limiter (N requests per minute).
10. Build a tiny in-memory key-value store with TTL.

2 Sample Challenge (with solution)

Find the first non-repeating character

```
from collections import Counter

def first_unique_char(s):
    count = Counter(s)
    for ch in s:
        if count[ch] == 1:
            return ch
    return None

print(first_unique_char("aabcbde"))
print(first_unique_char("aabbcc"))
```

Output:

```
d
d
None
```

Explanation:

- Count frequency of each character.
- Return the first character with frequency 1.
- If none found, return None.

3 Mini Project Ideas

1. Build a CLI To-Do app (using argparse, json).
2. File search tool (like grep) using generators.
3. Web scraper (requests + BeautifulSoup).
4. URL shortener (Flask + SQLite).
5. Simple chat server (sockets).
6. In-memory cache with TTL.
7. Log analyzer (parse large log files using generators).
8. API rate limiter (decorate Flask routes).

4 Tricky Interview Questions (Try Yourself)

1. Why does the below code print [3, 3, 3]?

```
funcs = []
for i in range(3):
    def f():
        return i
    funcs.append(f)
print([f() for f in funcs])
```

2. What will be the output?

```
a = [1, 2, 3]
b = a
b.append(4)
print(a)
```

3. Explain the difference:

```
a = (1, 2, 3)
b = (1, 2, 3)
print(a == b, a is b)
```

4. Why is this invalid?

```
d = {[1, 2]: "value"}
```

5. What will be printed?

```
def func(x, lst=[]):
    lst.append(x)
    return lst
print(func(1))
print(func(3))
```

5 Revision Checklist

- [ ] Python object model (id, type, value)
- [ ] Mutable vs immutable objects
- [ ] Namespaces and LEGB rule
- [ ] Functions and closures
- [ ] Iterators and generators
- [ ] Decorators and functools.wraps
- [ ] Concurrency, GIL, threading vs multiprocessing
- [ ] Asyncio and event loop (basics)
- [ ] Built-in types (list, dict, set, tuple, str, int)
- [ ] Hashing and hashable types
- [ ] Time complexity of common operations
- [ ] Common pitfalls and tricky questions
- [ ] Useful built-in functions and modules
- [ ] Memory management and garbage collection
- [ ] Practice coding problems
- [ ] Build a mini project

6 3-Month Learning Plan

Month 1: Core Concepts & Practice

- Revise all fundamentals (Pages 1–10).
- Solve 30+ coding problems.
- Implement small scripts and utilities.
- Focus on functions, iterators, decorators.

↓

Month 2: Internals & Advanced Topics

- Study concurrency, GIL, async (Page 11).
- Deep dive into built-in types (Page 12).
- Solve tricky interview questions (Page 13).
- Build 2–3 mini projects.

↓

Month 3: Application & Mock Interviews

- Revise quick sheet (Page 14).
- Solve 50+ mixed problems.
- Give mock interviews / explain code aloud.
- Build a final project of your choice.
- Make your own notes and cheatsheets.

7 Useful Resources

Official Docs	https://docs.python.org/3/
Free Tutorials	Corey Schafer (YouTube) Python Official Channel
Practice	LeetCode, HackerRank, Codeforces, NeetCode
Books	Python Crash Course Fluent Python (Advanced)
Real World	Build small projects Read open-source code Contribute to GitHub

8 Key Takeaways

- Everything in Python is an object.
- Understand mutability and object references.
- Functions are first-class objects.
- Use closures, decorators and generators to write clean and reusable code.
- GIL allows only one thread to execute Python bytecode at a time (use multipocessing for CPU-bound tasks).
- Lists, dicts and sets use dynamic arrays and hash tables.
- Know the time complexity of common operations.
- Be aware of common pitfalls (default arguments, late binding, mutable objects).
- Practice, build projects, and explain your code.
- Keep learning and stay curious!

9 Next Steps

- [ ] Solve the coding challenges above.
- [ ] Implement at least 2 mini projects.
- [ ] Make short notes for quick revision.
- [ ] Take mock interviews with friends.
- [ ] Explain concepts out loud (teaching).
- [ ] Read real-world Python code on GitHub.
- [ ] Keep practicing tricky questions.
- [ ] Apply these concepts in your work/projects.
- [ ] Follow @abhi_techhub for more such notes!

"Consistency today builds a better developer tomorrow."

